Derivation

03 November 2026 07:46

Goal:
Based on training data find the best possible model to classify future data points.

Unlike linear regression, we do not want a continuous linear function here, we want probabilities of a data point being in each group. To achieve this we use sigmoid function.

General process is similar to linear regression:

- Define a cost function with respect to our relevant parameters
- Apply gradient descent to minimize the cost function to find the optimal parameters.
- This requires finding the gradient of the loss function with respect to each parameter and taking a step in the opposite direction, size of the step is proportional to gradient and controlled by learning rate

Choosing a loss function:
We should not use Mean Square Error(MSE) like we did for gradient descent for many reasons, a major one being when using MSE with a sigmoid function you will create many local minimums, this means there is a chance gradient descent doesn't find the optimal global minimum.
Instead of MSE we use a cost function called "Cross-Entropy" which is designed for use with sigmoid functions.

Cross-Entropy can be divided into 2 separate functions, one for when y=0 and one for when y=1:

$h_\theta(x)$ is our models prediction: where $\theta = (w, b)$

$$Cost(h_\theta(x), y) = -\log(h_\theta(x)) \quad \text{if } y = 1$$

$$Cost(h_\theta(x), y) = -\log(1 - h_\theta(x)) \quad \text{if } y = 1$$

General:

$$Cost(h_\theta(x), y) = -\left[y \log(h_\theta(x)) + (1-y) \log(1 - h_\theta(x)) \right] \quad \text{where } \theta = (w, b)$$

This general formula lets us use the same formula for y=0 or y=1 as we multiply by y and (1-y) respectively so the irrelevant side cancels to 0.

Here is the equation for the loss function for our current model $h_\theta(x)$ over all points in a set:

$$-\frac{1}{n} \sum_{i=1}^{n} \left[y^i \log(h_\theta(x^i)) + (1-y^i) \log(1 - h_\theta(x^i)) \right] \quad \text{where } \theta = (w, b)$$

$$h_\theta(x) = h_{(w,b)}(x) = \frac{1}{1 + e^{-(wx+b)}}$$

Now we have defined our prediction model, sigmoid, and our cost function. What's left to do is optimize our weights, for this model "w" and "b", which again we will do by gradient descent. (There

are other methods however this is the one we will use for Logistic Regression, maybe in future tasks I will experiment with other optimisation algorithms)

To use gradient descent we must first differentiate our cost function.

Define:

$\hat{y}$: Our prediction from sigmoid

$$L = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$z = wx + b$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial b}$$

Loss differentiated with respect to both parameters, by chain rule

Computing:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}\left(-y\log(\hat{y}) - (1-y)\log(1-\hat{y})\right)$$

Where $\log(x) = \ln(x)$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

$$\frac{\partial}{\partial \hat{y}} = -\frac{c}{\hat{y}} + \frac{0}{1-\hat{y}}$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z}}\right) \quad \text{and} \quad \hat{y} = \frac{1}{1+e^{-z}}$$

$$= \frac{\partial}{\partial z}\left(1+e^{-z}\right)^{-1}$$

$$= -\left(1+e^{-z}\right)^{-2}\left(-e^{-z}\right)$$

$$= \frac{e^{-z}}{\left(1+e^{-z}\right)^2} \quad \text{Now we should define in terms of } \hat{y}$$

$$= \frac{1}{1+e^{-z}} \times \frac{e^{-z}}{1+e^{-z}} = \hat{y} \times \frac{e^{-z}}{1+e^{-z}}$$

$$= \hat{y} \times \frac{(1+e^{-z})-1}{1+e^{-z}}$$

$$= \hat{y} \times \left(1 - \frac{1}{1+e^{-z}}\right) = \hat{y}\left(1-\hat{y}\right)$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}\left(1-\hat{y}\right)$$

$$\frac{\partial z}{\partial w} = \frac{\partial}{\partial w} wx+b = x$$

$\therefore$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w}$$

$$= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \times \hat{y}(1-\hat{y}) \times x$$

$$= \left(-y(1-\hat{y}) + \hat{y}(1-y)\right) \times x$$

$$= \left(-y + \hat{y}y + \hat{y} - \hat{y}y\right) \times x$$

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

For b all is same except $\frac{\partial z}{\partial b} = 1$

$\therefore$

$$\frac{\partial L}{\partial b} = (\hat{y} - y)$$

Now we have our gradients of loss for each parameter, so we can finally write our update rules

$$w_{new} = w_{old} - \left(\left(\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)x_i\right) \times \text{learning rate}\right)$$

$$b_{new} = b_{old} - \left(\left(\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\right) \times \text{learning rate}\right)$$

Also note that you can include "b" in the "w" vector by making w_1=b and enforcing x_1=1, in this case the entire update rules is:

$$w_{new} = w_{old} - \left(\left(\frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_i \right) \times \text{learning rate} \right)$$

Now we have our update rule it is clear how to implement this:
Set a base "w" matrix
Define a predict() function which uses the sigmoid with our weights to predict, this prediction becomes y hat
Loop the update function through all data points
Repeat until our update is very small, below a certain threshold, or a certain number of steps has been reached.